05.JavaScript & DOM

19 octobre 2026

Développement web il3

JavaScript & DOM

HE-Arc (DGR) 2026

JavaScript hier

- Page web = HTML (+ CSS + JavaScript)
- Exécuté par le browser (client)
- Interprété, faiblement typé, OO
- Historiquement
  - Depuis Netscape 2 (1995, Brendan Eich)
  - Petites applications exécutées par le navigateur
  - DHTML : rollovers, validation de formulaires, …

JavaScript aujourd'hui

- Page web = HTML + CSS + **JavaScript**
- Compilation JIT
- HTML5, AJAX, bookmarklets
- Userscripts [1] : Tampermonkey [2], Brave Custom scriptlets [3]
- One Page Apps
- Implémentations hors-browser
  - Node.js, Spidermonkey, Rhino

1. https://github.com/awesome-scripts/awesome-userscripts
2. https://www.tampermonkey.net/
3. https://brave.com/privacy-updates/32-custom-scriptlets/

  - script d'app (Qt, Notepad++, ...)
- Langage cible de compilateurs : emscripten [4], WebAssembly [5]
- Embarqué : Espruino [6], robotique : Node Bots [7], [Johnny 5][https://johnny-five.io/]
- Applications Desktop : Electron [8], sciter [9], [Tauri][https://tauri.app/] (+ [Pake][https://pake.run/])

*Script

- ECMAScript : Norme depuis 1997
  - Juin 2026 : ECMA-262 17th edition [10]
  - Support [11] des différentes implémentations
  - Conversions avec BabelJS [12]
- JavaScript : implémentation Firefox (réf. MDN)
- Variantes (à transpiler) :
  - Typescript [13] :
    - variante fortement typée (MS)
    - plus maintenable, recommandé par défaut !
    - **trans**pilé -> JS
    - Développement *symbiotique* avec Monaco / VSCode [14]

JavaScript

- Différentes implémentations [15] : navigateur, srv, apps, ...
- Permissif : du mauvais code est peu maintenable
  - Design Patterns [16], book [17]
  - Bonnes pratiques [18], W3S [19]
- Interface pour scripter le navigateur
  - Accès et modification du contenu via DOM

4. https://emscripten.org/
5. http://webassembly.org/
6. http://www.espruino.com/
7. https://nodebots.io/
8. https://electronjs.org/
9. https://sciter.com/
10. https://www.ecma-international.org/publications-and-standards/standards/ecma-262/
11. https://compat-table.github.io/compat-table/
12. https://babeljs.io/
13. https://www.typescriptlang.org/
14. https://www.youtube.com/watch?v=kHL3XzjpT5w
15. https://en.wikipedia.org/wiki/List_of_ECMAScript_engines
16. https://www.patterns.dev/
17. https://patterns.addy.ie/
18. https://good.js.org/
19. https://www.w3schools.com/js/js_best_practices.asp

  - Bookmarklets [20], A11y [21], Contrast [22], dev tools [23] exemples plus vieux [24]
  - Requêtes HTTP (Fetch API, Xml Http Request)
- Développement d'applications complètes, parfois offline
- Langage de script généraliste (paquets npm)

Caractéristiques du langage

- Orienté Objet par prototype
- Syntaxe proche de C, Java
- Faiblement typé :
  - Pas de déclaration, type déterminé par la dernière affectation
  - Risque : typo => nouvelle variable. Utiliser `const` et `let`
- Types :
  - Primitifs : `Boolean Null Undefined Number String Symbol`
  - Objets : `Object Function`
- Particularités
  - Prototypes [25]
  - Fermetures [26]
  - Promesses [27] (MDN [28], Google [29])

Fonctions

- Pas de type de retour
- Possibilité de retourner ou non une valeur
- Sans retour, valeur spéciale : undefined
- Pas de surcharge (la dernière définie prime)
- `function` est un type
- Fonctions imbriquées, anonymes
- Fonctions globales :

20. https://bookmarklet.io/library
21. https://a11y-tools.com/bookmarklets/
22. https://webaim.org/resources/contrastchecker/bookmarklet
23. https://bookmarklets.app.quest/
24. http://www.hongkiat.com/blog/100-useful-bookmarklets-for-better-productivity-ultimate-list/
25. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Le_mod%C3%A8le_objet_JavaScript_en_d%C3%A9tails
26. http://www.w3schools.com/js/js_function_closures.asp
27. https://www.promisejs.org/
28. https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Promise
29. https://web.dev/articles/promises?hl=fr

```
escape(), unescape(), isFinite(), isNaN(),
parseFloat(), parseInt(), Number(), String(),
eval(), ...
```

JavaScript dans la page web

- Éléments <script> exécutés dans l'ordre de la page
- Evénements (onclick, onerror, onsubmit, ...)
  - Embarqués dans les balises (onXXX)

```
<div id="intro" onclick="change();" />
```

Utiliser DOM

```
<script type="text/javascript">

    document.getElementById("intro").onclick = change;

</script>
```

- Conseillé d'inclure le code (attribut src)

```
    <script type="module" src="app.js"></script> // pour un module ESM
    <script defer src="ancien-script.js"></script> // pour les autres scripts
```

`language="JavaScript"` est déprécié et `type` vaut par défaut `text/javascript`.

> The type attribute gives the language of the script or format of the data. [...] The default, which is used if the attribute is absent, is "text/javascript".
> HTML5 : script [30]

Unobstrusive JS [31] (Pourquoi en 2026 [32] ?)

- Séparation JS...

```
document.addEventListener("DOMContentLoaded", function() {
    document.getElementById('date').addEventListener("change", validateDate);
};
```

- ...et HTML

30. https://www.w3.org/TR/html5/scripting-1.html#the-script-element
31. https://en.wikipedia.org/wiki/Unobtrusive_JavaScript
32. https://www.perplexity.ai/search/a62728eb-54cf-4ba0-85a4-d6f3287ed6bb

```
<input type="text" name="date" id="date" />
```

- Dégradation élégante
  - Fonction critique pas JS only si alternative HTML/CSS possible
- Accessibilité
  - Les fonctionnalités restent accessibles en cas d'erreur
- Utilisabilité
  - Le script doit faire gagner du temps, pas distraire

> It is an incredibly popular mistake to use `load` where `DOMContentLoaded` would be much more appropriate, so be cautious.
>
> MDN : DOMContentLoaded [33]

Node.js [34] / Deno [35] (Cloudflare [36])

- Node.js : une implémentation hors navigateur
  - environnement d'exécution + bibliothèques
  - event driven, non-blocking IO -> scalable
  - V8 engine
  - scripts exécutables sans navigateur
  - npm [37] : gestionnaire de paquets
  - gulp : make js
- Exemples [38] d'applications
  - gulp, grunt, bower, yarn
  - browserify
  - serveur http
  - axios, express, cordova, forever, dev, pm2, karma, sails, phantomjs
- Tuto [39], Playground [40]

DOM

- Document Object Model
- Représentation arborescente de la page
- Accessible depuis objet JS document

33. https://developer.mozilla.org/en/docs/Web/Events/DOMContentLoaded
34. https://nodejs.org
35. https://www.reddit.com/r/node/comments/nx9qqr/deno_vs_nodejs_a_comparison_you_need_to_know/
36. https://deno.com/blog/cloudflare
37. https://www.npmjs.com
38. https://github.com/whokartikgambhir/Open-Source-node-backend-Apps
39. https://www.tutorialspoint.com/nodejs/index.htm
40. https://stackblitz.com/edit/node-js-playground?file=index.js

- Possibilité d'accéder au contenu de la page :
  - Lecture
  - Modification
  - Ajout
- JS peut donc modifier le contenu d'une page

DOM [41]

```
<html>
<head>
   <title>My title</title>
</head>
<body>
    <h1>A heading</h1>
    <a href="#">Link text</a>
</body>
</html>
```

L'objet Document

- Trouver ou modifier des éléments
- Méthodes de `Document`

```
querySelector(), querySelectorAll(),
getElementById(), getElementsByTagName(), getElementsByClassName(),
createElement(), createTextNode()
```

- Méthodes de `Node` (appel depuis nœud parent)

```
insertBefore(child), appendChild(child),
removeChild(child), replaceChild(new,old)
```

Ajouter un noeud

```
function addNode() {
     var inText = document.getElementById("textArea").value;
     var newText = document.createTextNode(inText);
```

41. http://bioub.github.io/dom-visualizer/

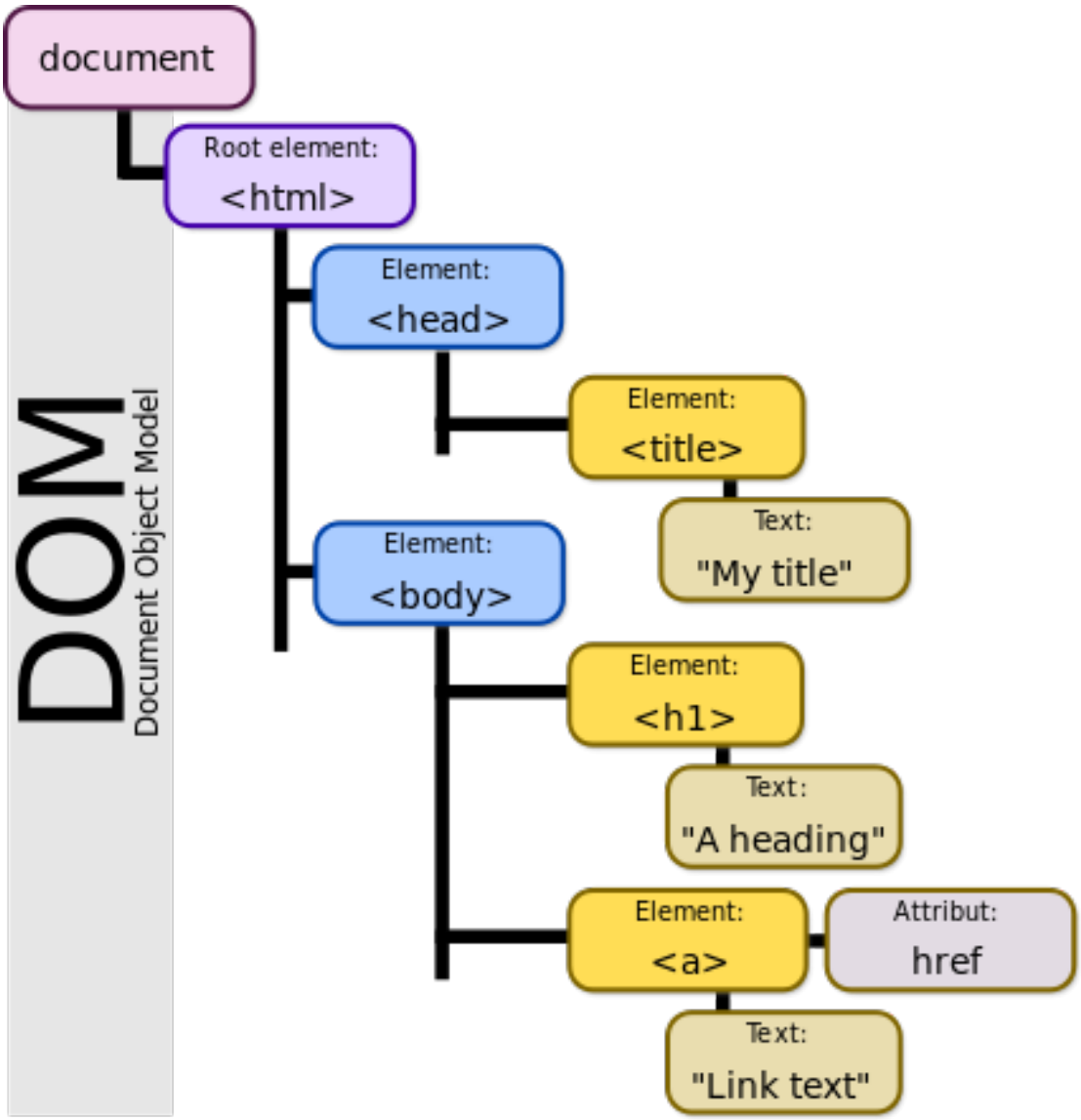


Figure 1 – DOM tree

```
    var newGraf = document.createElement("p");
    newGraf.appendChild(newText);

    var docBody = document.getElementsByTagName("body")[0];
    docBody.appendChild(newGraf);
}
```

- Création du nouveau nœud :
  - `newText` contient le texte à ajouter
  - `newGraf` est un élément `p` qui contient le texte
- Ajout du nœud comme une feuille de body :
  - Sélection du parent (le premier noeud `body`)
  - Ajout du nouveau nœud depuis son parent

Supprimer un nœud

```
function delNode() {
   var allGrafs = document.getElementsByTagName("p");

   if (allGrafs.length > 1) {
      var lastGraf = allGrafs.item (allGrafs.length-1);
      lastGraf.parentNode.removeChild(lastGraf);
   }
   else {
      console.error("Nothing to remove!");
   }
}
```

- Sélection du nœud à supprimer :
  - `allGrafs` contient tous les éléments `p`
  - `lastGraf` contient le denier du tableau `allGrafs`
- Suppression :
  - Suppression du nœud sélectionné depuis son parent [42]

Insérer un nœud

```
function insertNode() {
    var newText = document.createTextNode("New Text");
    var newGraf = document.createElement("p");
    newGraf.appendChild(newText);
```

42. https://developer.mozilla.org/en-US/docs/Web/API/Node/parentNode

```
    var divMod = document.getElementsByTagName("div")[0];
    var allGrafs = divMod.getElementsByTagName("p");
    var oldGraf = allGrafs.item(0);              // position

    divMod.insertBefore(newGraf,oldGraf);
}
```

- Création du nouveau nœud :
  - `allGrafs` contient tous les éléments `p`
  - `lastGraf` contient le denier du tableau `allGrafs`
- Insertion :
  - Recherche du parent
  - Recherche du frère gauche
  - Insertion depuis le parent

Avec jQuery

- Création et ajout :

```
var noeud = $('<p>Nouveau texte</p>');  // create node
$("body").append(noeud);                // après le dernier fils
```

- Sélection et Suppression :

```
var noeud = $("p"); // select node(s)
noeud.remove();
```

Références

- Une réintroduction à JavaScript [43]
- Things to avoid in JS [44]
- How does it feel to learn JS in 2016 [45], What to know in JS, 2026 [46]
- Référence MDN [47]
- Tutoriels The Modern JS Tuto [48], w3schools [49], LearnJS [50]

43. https://developer.mozilla.org/fr/docs/Web/JavaScript/Une_r%C3%A9introduction_%C3%A0_JavaScript
44. https://waspdev.com/articles/2025-06-13/things-to-avoid-in-javascript
45. https://hackernoon.com/how-it-feels-to-learn-javascript-in-2016-d3a717dd577f
46. https://blog.master.dev/what-to-know-in-javascript-2026-edition/
47. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference
48. https://javascript.info/
49. http://www.w3schools.com/js/
50. https://learnjavascript.online/

- devtools Chrome et Firefox (F12, Ctrl+Shift I)
- Visualisation du DOM [51]
- Outils web
  - JSFiddle [52]
  - JSLint [53]

Sources

51. http://bioub.github.io/dom-visualizer/
52. https://jsfiddle.net/
53. http://www.jslint.com/